Smart Budgeting and Financial Goal Management System

Test Plan, Implementation Roadmap & Risk Register

Project	Smart Budgeting and Financial Goal Management System — Smart Personal Budgeting and Financial Goal Management System
Prepared for	Team J
Version / Date	Version 1.0 | 8 October 2026

Working draft for project implementation. Items marked "decision required" should be confirmed by the team before development.

1. Testing strategy

Testing should be continuous: verify each feature before integrating it, then test end-to-end flows and privacy boundaries.

Test level	Purpose	Example
Unit tests	Verify small calculations and validation rules.	Budget remaining = limit − matching expenses.
API/integration tests	Verify API, database and permissions work together.	User A cannot retrieve User B's transaction.
UI tests	Verify forms, navigation and feedback.	Invalid amount produces a visible validation message.
System/end-to-end tests	Verify complete user journeys.	Register → add records → create budget → view report.
Acceptance tests	Confirm the SRS requirements are met.	All acceptance criteria in SRS pass.

2. Core test cases

ID	Test scenario	Expected result	Result
TC-01	Registration with valid details	Account created and can sign in.	Pass/Fail
TC-02	Register using duplicate identifier	Clear validation error; no duplicate account.	Pass/Fail
TC-03	Sign in with wrong password	Access denied; no private data shown.	Pass/Fail
TC-04	Add income with positive amount	Record saved and income total increases by exact amount.	Pass/Fail
TC-05	Add expense with zero/negative amount	Request rejected; no record saved.	Pass/Fail
TC-06	Edit/delete an owned transaction	Only selected record changes/removes; summaries recalculate.	Pass/Fail
TC-07	Create budget and add matching expense	Used/remaining values update correctly.	Pass/Fail
TC-08	Expense outside budget date range	Expense does not affect that budget.	Pass/Fail
TC-09	Create goal and contribution	Progress equals contribution total / target × 100, capped for display.	Pass/Fail
TC-10	Filter report by dates	Only records inside the chosen range are included.	Pass/Fail
TC-11	Attempt cross-user object access	Server returns denied/not found and data remains unchanged.	Pass/Fail
TC-12	Refresh after saving	Saved data remains available.	Pass/Fail
TC-13	No records yet	Dashboard shows	Pass/Fail

		zero/empty states without crashing.	
TC-14	Invalid report date range	Clear error; no misleading report is shown.	Pass/Fail
TC-15	Responsive mobile layout	Core screens remain usable at mobile width.	Pass/Fail

3. Suggested implementation roadmap

Phase	Work items	Deliverable / exit condition
0. Confirm scope	Approve MVP, stack, login identifier, currency, budget periods, contribution model.	Signed-off SRS and decisions list.
1. Design	Finalize use cases, ERD, wireframes, API contract and repository structure.	Design reviewed by team.
2. Foundation	Set up Git, frontend/backend projects, database, environment variables, migrations and authentication.	App runs locally; user accounts work.
3. Transactions	Build categories and income/expense CRUD with validation and filters.	Records persist and tests pass.
4. Budgets & goals	Build budget calculations, savings goals and contributions.	Manual calculations match app results.
5. Dashboard & insights	Add cards, charts, reports and transparent rule-based insights.	Dashboard reconciles to records.
6. Security & quality	Test ownership isolation, input validation, errors, responsive design and backups.	Critical tests pass; no known cross-user access issue.
7. Deployment & handover	Deploy, document setup, demo with fictional data, prepare final report.	Working demo and setup guide delivered.

4. Risk register

Risk	Likelihood / impact	Mitigation
Scope expands beyond available time	High / High	Keep bank integration and AI out of MVP; use change requests.
Incorrect financial calculations	Medium / High	Write calculation rules first; use unit tests and manual reconciliation.
Private data exposed between users	Medium / Very High	Server-side ownership checks; test every endpoint with two users.
Team coordination / merge conflicts	Medium / Medium	Assign feature ownership, use branches and regular integration.
Data loss	Low-Medium / High	Database backups and restore test

		before deployment.
Unclear budget/goal semantics	High / Medium	Resolve open decisions before database implementation.
Poor mobile usability	Medium / Medium	Use responsive layouts and test on narrow screens.

5. Suggested team roles

- Project coordinator / analyst: maintain scope, requirements, meeting notes and acceptance checklist.
- Frontend developer: screens, navigation, forms, dashboard and charts.
- Backend developer: API, authentication, business logic and permissions.
- Database / QA lead: schema, migrations, test cases, calculation checks and test evidence.
- Deployment / documentation lead: environment setup, deployment, user guide and presentation.

These are suggested responsibilities, not assumptions about current Team J membership. One person may hold more than one role.

6. Definition of done

- Feature matches the approved requirement and use case.
- Input validation and permission checks are implemented.
- Relevant tests pass and test evidence is recorded.
- UI includes loading, empty and error states where appropriate.
- Code is committed with a meaningful message and reviewed by another team member where possible.
- Documentation is updated and the feature works in the integrated application.